Complemento Práctico: El Scanner y la Prioridad Matemática

Solución de errores comunes en tiempo de ejecución

En esta unidad, el PDF nos enseña la sintaxis para leer datos y realizar cálculos. Sin embargo, en la práctica profesional nos encontramos con dos obstáculos recurrentes que no siempre son errores de sintaxis (el código compila bien), sino errores de **lógica** o de **comportamiento del entorno**. Aquí analizaremos por qué Java a veces "salta" preguntas al usuario y por qué nuestras cuentas matemáticas pueden dar resultados diferentes a los de una calculadora estándar.

Contenido

Contexto

¿Por qué leer datos en Java es tan "verboso" comparado con otros lenguajes?

El "Bug" del Scanner

El problema del salto de línea fantasma.

Matemática Computacional

La trampa de la precedencia de operadores.

La Complejidad de la Entrada de Datos

En el video mencionamos que "Java no es el mejor lenguaje para aprender desde cero" debido a su verbosidad. Mientras que en lenguajes como Python leer un dato es una sola palabra (input()), en Java necesitamos:

1. Importar una librería

java.util.Scanner

2. Instanciar un Objeto

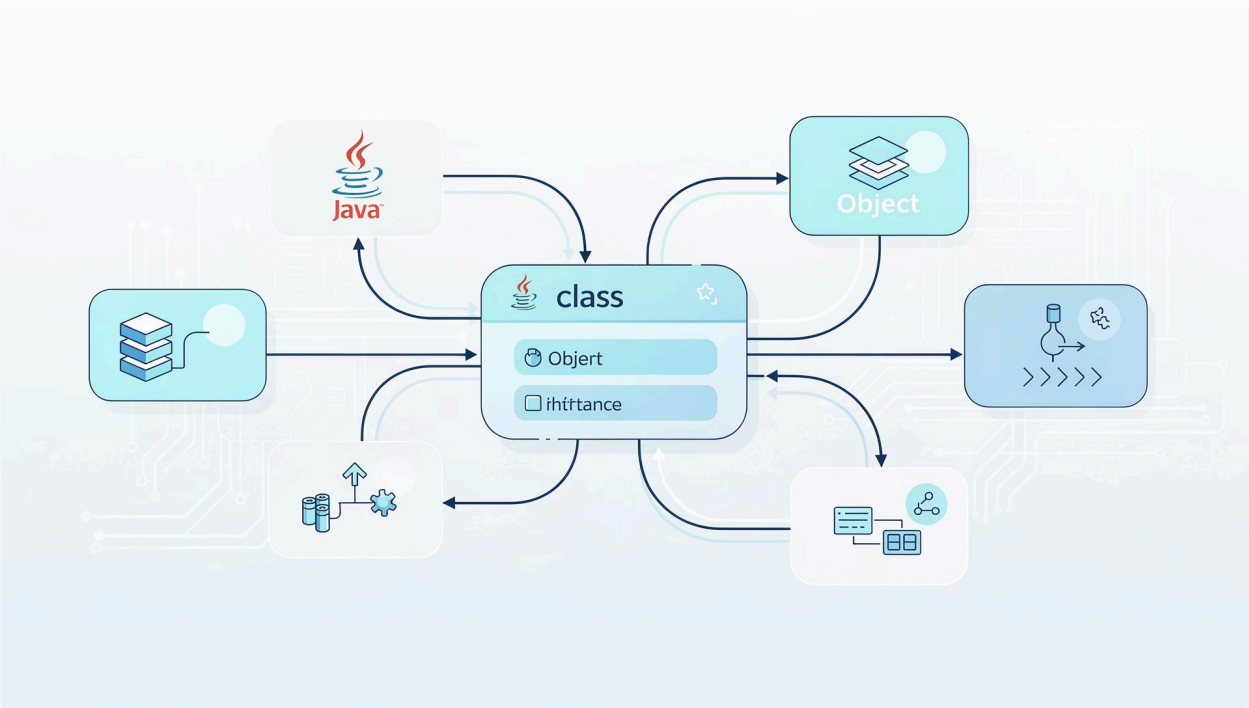
new Scanner

3. Definir el flujo de entrada

System.in

¿Por qué tanta burocracia?

Porque Java es **Estrictamente Orientado a Objetos**. No existen funciones "sueltas" o mágicas. Todo debe pasar por un objeto responsable. En este caso, el objeto Scanner es un "escáner" complejo capaz de analizar flujos de texto, archivos o conexiones de red, no solo el teclado. Esa potencia conlleva una sintaxis más pesada.



El Error Invisible: Cuando el Scanner "se salta" pasos

Este es el error más consultado en los foros de la materia. Ocurre cuando mezclamos la lectura de números (int, double) con la lectura de texto (String).

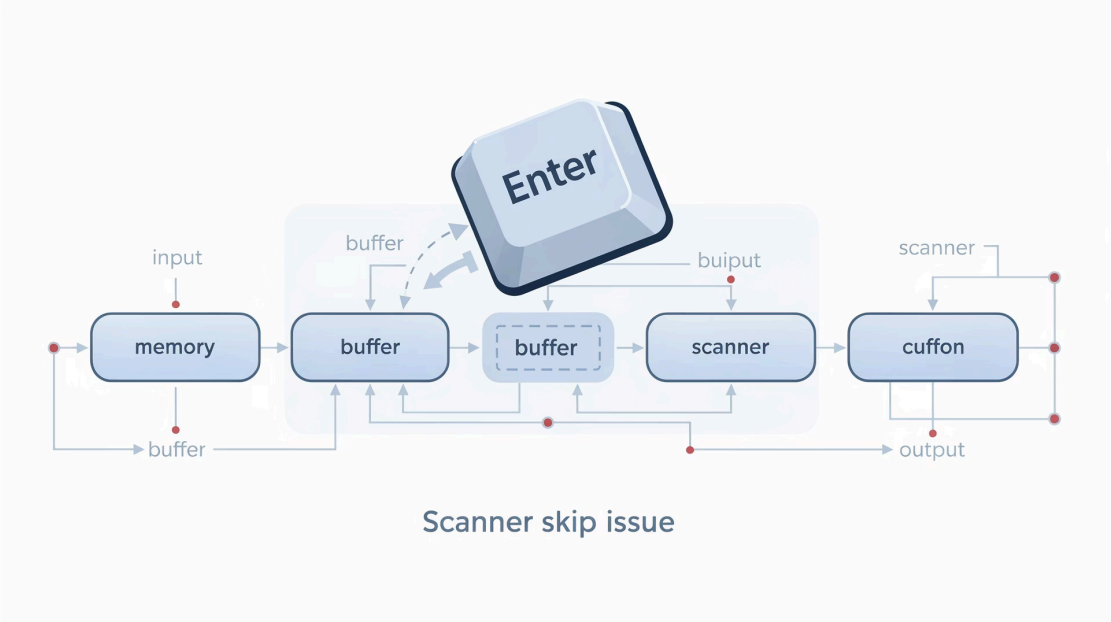
El Escenario

Queremos pedir una edad y luego un nombre.

```
System.out.print("Edad: ");
int edad = sc.nextInt();
System.out.print("Nombre: ");
String nombre = sc.nextLine(); // ¡El programa se salta esto!
```

El Diagnóstico

El método nextInt() lee el número (ej: "25"), pero **no consume el "Enter"** que el usuario presionó para confirmar. Ese "Enter" (\n) queda flotando en el buffer (memoria temporal de entrada). Cuando llega la siguiente instrucción nextLine(), esta ve el "Enter" flotando, piensa que el usuario ya escribió una línea vacía y finaliza inmediatamente.



Solución Parche: Limpieza Manual de Buffer (No recomendada)

Para solucionar el problema del "salto de línea", históricamente se enseñaba a realizar una "limpieza manual" del buffer agregando un nextLine() vacío cada vez que se cambiaba de leer números a leer texto.

⚠ Aclaración importante: Aunque esta técnica soluciona el bug temporalmente, **no es el estándar que evaluaremos en Programación 2**.

```
// FORMA TRADICIONAL (Desaconsejada en esta cátedra)
System.out.print("Edad: ");
int edad = sc.nextInt(); // Deja el "Enter" en el buffer

// Limpieza manual obligatoria (código extra que ensucia la lógica)
sc.nextLine();

System.out.print("Nombre: ");
String nombre = sc.nextLine();
```

Estrategia Oficial de la Cátedra: Parseo de Datos (Estándar Evaluado)

El problema del buffer sucio ocurre principalmente porque mezclamos métodos que leen "tokens" (como nextInt() o nextDouble()) con aquellos que leen líneas completas (nextLine()). Los métodos de token dejan el carácter de "Enter" (\n) en el buffer, lo que sabotea la siguiente lectura.

Propuesta Profesional (obligatoria para aprobar las rúbricas de esta materia)

La solución es simple y radical: erradicar el uso de nextInt() o nextDouble(). En su lugar, utilizaremos SIEMPRE nextLine() para toda entrada de datos y delegaremos la conversión a las clases envolventes (Wrappers).

- Si queremos solicitar un texto: `String nombre = sc.nextLine();`
- Si necesitamos un número, lo leemos y convertimos (parseamos) en la misma línea: `int edad = Integer.parseInt(sc.nextLine());`

La gran ventaja de este enfoque es que nextLine() siempre consume todo, incluido el carácter de "Enter". El buffer queda impecable, eliminando por completo los saltos de línea fantasma y reduciendo la cantidad de código escrito.

Comparativa de Enfoques

❌ FORMA TRADICIONAL (Propensa a errores y penalizada en evaluación)

```
System.out.print("Edad: ");
int edad = sc.nextInt(); // ¡Riesgo! Deja el 'Enter' en el buffer.

System.out.print("Nombre: ");
// Si no pusiste un sc.nextLine() extra antes, esta línea fallará y se saltará.
String nombre = sc.nextLine();
```

✅ FORMA PROFESIONAL (Estándar Seguro de Programación 2)

```
// 1. Leemos y parseamos en un solo movimiento limpio:
System.out.print("Edad: ");
int edad = Integer.parseInt(sc.nextLine()); // Consume el número y el 'Enter'
simultáneamente.

// 2. El buffer está limpio y listo para usarse sin código extra.
System.out.print("Nombre: ");
String nombre = sc.nextLine(); // Espera correctamente la entrada del
usuario.
```

Creando Wrappers de Lectura (Anticipo a la Unidad 2)

Para no repetir el código de conversión (Integer.parseInt(), Double.parseDouble()) en todas partes y manejar posibles errores (como escribir "veinte" en lugar de "20", o usar una coma en lugar de un punto), es una excelente práctica crear nuestros propios métodos utilitarios.

> *Aviso: Aunque la teoría completa sobre la creación, estructuración y alcance de estos métodos será el foco central de la **Unidad 2: Funciones**, introducimos aquí este concepto para que comiences a programar de forma robusta desde el primer día.*

Estos métodos actúan como un "wrapper" o envoltorio, centralizando la compleja lógica de validación, limpieza de buffer y conversión, haciendo que nuestro código principal (main) sea limpio y fácil de leer.

Beneficios de un Wrapper de Lectura Segura

Reutilización de Código (DRY)

Evita duplicar la lógica de conversión y el manejo de excepciones cada vez que necesitas pedir un dato.

Manejo de Errores Robusto

Captura automáticamente errores de formato y guía al usuario para que proporcione una entrada válida sin que el programa colapse ("crashee").

Buffer Siempre Limpio

Al utilizar consistentemente nextLine() dentro del wrapper, se elimina por completo el problema del "Enter" residual.

Adaptabilidad (UX)

Permite corregir errores comunes del usuario antes de que Java los procese (como cambiar comas por puntos en los decimales).

Código del Módulo: Herramientas de Lectura

```
// Wrapper para leer Decimales (Soporta coma y punto)
public static double leerDouble(Scanner sc, String mensaje) {
    double numero = 0;
    boolean datoValido = false;
    do {
        try {
            System.out.print(mensaje);
            String textoIngresado = sc.nextLine();
            // Reemplazamos coma por punto para evitar el error de formato en Java
            String textoNormalizado = textoIngresado.replace(',', '.');
            numero = Double.parseDouble(textoNormalizado);
            datoValido = true;
        } catch (NumberFormatException e) {
            System.out.println("Error: Ingrese un número decimal válido (ej: 75.5 o
75,5).");
        }
    } while (!datoValido);
    return numero;
}
```

Uso en el programa principal:

Ahora, pedir datos complejos es tan simple y seguro como escribir:

```
double peso = leerDouble(sc, "Ingrese su peso en kg: ");
```

Validación de CUIT Argentino

En el ámbito de los sistemas de gestión, la integridad y el formato correcto de los datos son cruciales. Un ejemplo fundamental es el Código Único de Identificación Tributaria (CUIT) en Argentina. No podemos permitir que el sistema acepte cualquier secuencia de caracteres; este debe adherirse estrictamente a un formato específico: once dígitos numéricos y sin guiones. Esta validación inicial no solo garantiza la coherencia de la información, sino que también es el primer paso para procesar correctamente transacciones fiscales y administrativas, evitando errores y simplificando el posterior manejo de datos.

Código del Módulo

```
public static String leerCUIT(Scanner sc) {
    String cuit = "";
    boolean valido = false;

    do {
        System.out.print("Ingrese CUIT (11 números sin guiones): ");
        cuit = sc.nextLine();

        // Validamos longitud y que sean solo números (Regex)
        if (cuit.matches("[0-9]{11}")) {
            valido = true;
        } else {
            System.out.println("Error: El CUIT debe tener 11 dígitos numéricos.");
        }
    } while (!valido);

    return cuit;
}
```

Validación de Correo Electrónico

La validación de un correo electrónico puede ser un desafío sorprendentemente complejo si se busca una perfección absoluta, debido a la gran variedad de formatos válidos según los estándares RFC (Request for Comments). Sin embargo, para la mayoría de las aplicaciones, podemos implementar reglas de negocio prácticas que aseguren un formato básico y aceptable. El objetivo principal es garantizar que la dirección contenga el carácter arroba (@) que separa el nombre de usuario del dominio, y que no existan espacios en blanco, que son intrínsecamente inválidos en una dirección de correo.

Este tipo de validación inicial no solo ayuda a mantener la calidad de los datos en nuestros sistemas, sino que también mejora la experiencia del usuario al proporcionar retroalimentación inmediata sobre errores comunes. Al implementar estas verificaciones tempranas, evitamos que datos malformados se propaguen por la aplicación, simplificando futuras operaciones como el envío de notificaciones o la gestión de cuentas.

Código del Módulo para Validación Básica de Email

```
public static String leerEmail(Scanner sc) {
    String email = "";
    boolean emailValido = false;

    do {
        System.out.print("Ingrese su Email: ");
        email = sc.nextLine();

        // Aplicamos reglas básicas de negocio: debe contener '@' y no tener espacios
        if (email.contains("@") && !email.contains(" ")) {
            emailValido = true; // El formato básico es aceptado
        } else {
            System.out.println("Error: Formato de correo inválido. Asegúrese de incluir '@' y no usar
espacios.");
        }
    } while (!emailValido);

    return email;
}
```

Validación de Teléfono (+54 9)

Para asegurar la consistencia y la interoperabilidad de los datos en sistemas empresariales, la normalización de números de teléfono a un formato internacional estandarizado es fundamental. Este requisito es común en muchas empresas, especialmente cuando manejan clientes o contactos de diferentes regiones, como es el caso de Argentina con el formato `+54 9`. Implementaremos una validación estricta para garantizar que el usuario ingrese un número de móvil respetando el prefijo internacional y el número de dígitos exacto. Esto no solo mejora la calidad de la base de datos, sino que también facilita la comunicación y evita errores en futuros procesos automatizados, asegurando que cada contacto sea siempre accesible.

Código del Módulo para Teléfono Argentino

```
public static String leerTelefono(Scanner sc) {
    String telefono = "";
    // Regex: Empieza con +54 9, un espacio, y luego 10 dígitos (para número completo de
Argentina).
    // Ejemplo: +54 9 2611234567
    String patron = "^\\+54 9 \\d{10}$";

    do {
        System.out.println("Formato requerido: +54 9 2611234567 (ejemplo para Mendoza)");
        System.out.print("Ingrese Teléfono: ");
        telefono = sc.nextLine();

        if (telefono.matches(patron)) {
            break;
        }
        System.out.println("Error: Por favor, respete el formato internacional (+54 9 XXXXXXXXXX).");

    } while (true);

    return telefono;
}
```

Este método proporciona una forma robusta y sencilla de garantizar que los números de teléfono ingresados cumplan con un formato específico, crucial para bases de datos limpias y operaciones de comunicación efectivas. Al usar expresiones regulares, podemos definir patrones complejos con gran precisión y validar la entrada del usuario de manera eficiente.

Ventajas de la Modularización

Al implementar métodos dedicados para la lectura y validación de datos, como los ejemplos de `leerEntero`, `leerCUIT`, y `leerEmail`, logramos una serie de objetivos fundamentales en la ingeniería de software que van más allá de la mera funcionalidad. Estos pilares mejoran significativamente la calidad, mantenibilidad y robustez de nuestro código.

Abstracción

Nuestro método `main` o cualquier otra función que necesite interactuar con el usuario para obtener datos, se mantiene limpio y legible. Se abstrae de los detalles de la validación y el manejo de errores de entrada, llamando simplemente a una función específica que se encarga de todo el proceso complejo internamente.

Robustez

El programa adquiere una mayor resistencia a entradas no válidas. Ya no colapsa o produce errores inesperados si el usuario ingresa datos que no corresponden al formato esperado (ej. texto en lugar de números), garantizando un comportamiento predecible y estable en todo momento.

Limpieza

Evitamos la necesidad de instrucciones repetitivas como `sc.nextLine()` para limpiar los buffers del scanner, un problema común y fuente de muchos dolores de cabeza. Nuestra estrategia de "leer todo como String" y luego intentar la conversión, simplifica el manejo del buffer de entrada de forma natural.

Para aprovechar al máximo estos beneficios, nuestra recomendación es que copien estos métodos de utilidad en una clase auxiliar dedicada, por ejemplo, `Validadores.java`. Al centralizar estas funciones de validación, podrán reutilizarlas fácilmente en todos sus Trabajos Prácticos y proyectos futuros, fomentando así una cultura de código limpio y eficiente.

La Trampa de la Precedencia

En el video planteamos un desafío rápido: ¿Cuánto es 2 + 3 * 4?

Respuesta intuitiva

(Izquierda a derecha)

2 + 3 = 5, luego 5 * 4 = **20**.

Respuesta Java

(y matemática real)

14.

Jerarquía de Operadores

Los lenguajes de programación no leen estrictamente de izquierda a derecha. Leen por **prioridad**.

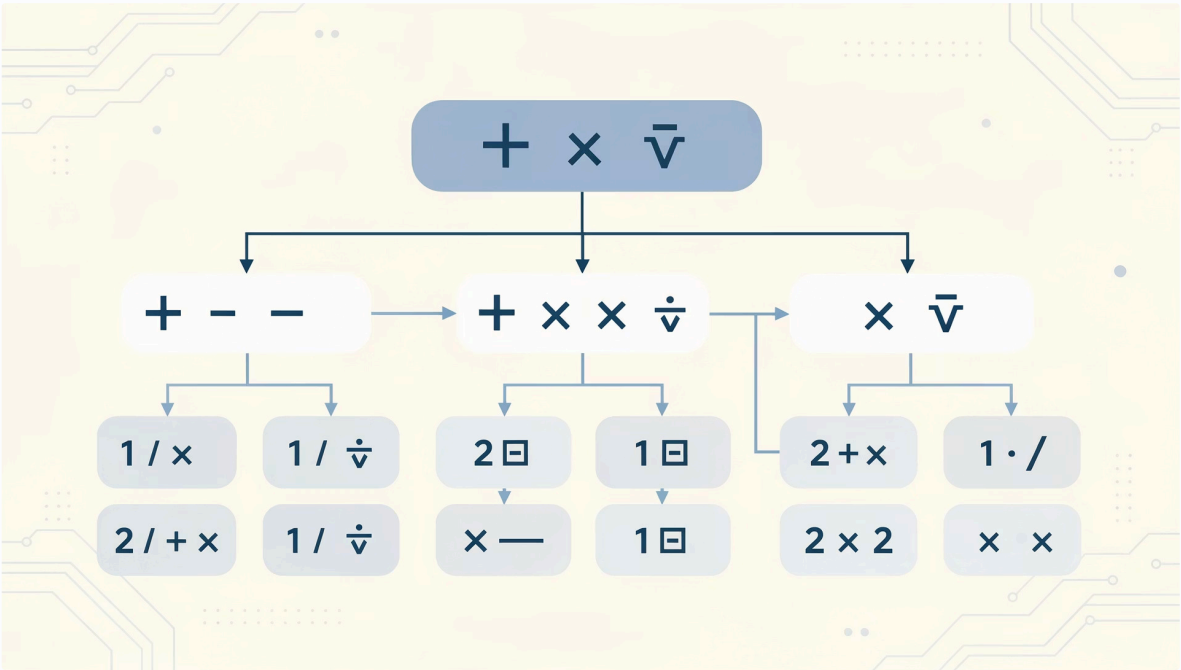
Prioridad 1

Multiplicación *, División / y Módulo %.

Prioridad 2

Suma + y Resta -.

Por lo tanto, Java primero ve el 3 * 4 (12) y luego le suma 2. Si su programa calcula facturas o promedios ignorando esto, tendrán errores financieros graves.



Forzando la Prioridad

Si nosotros *realmente* queremos que la suma se haga antes, no podemos confiar en el orden de escritura. Debemos ser explícitos.

El Poder de los Paréntesis

Al igual que en álgebra, los paréntesis () tienen la prioridad máxima absoluta. Rompen cualquier jerarquía preestablecida.

Prioridad por defecto

(Multiplicación gana)

```
int resultado1 = 2 + 3 * 4; // 14
```

Prioridad forzada

(Paréntesis gana)

```
int resultado2 = (2 + 3) * 4; // 20
```

Consejo Profesional

Incluso cuando no sea estrictamente necesario, **usen paréntesis** si la fórmula es compleja. Esto no solo asegura el cálculo, sino que facilita la lectura a otros programadores que revisen su código. Es mejor escribir a + (b * c) que a + b * c, aunque el resultado sea el mismo, porque elimina la ambigüedad visual.